

World 8 / Z0-A - COMPLETE ARCHIVAL ARCHITECTURE MASTER

Provider-independent persistent runtime with independent observation and proposal-only development

Status: FINAL DESIGN BASELINE / NOT PRODUCTION / ARCHIVAL MASTER

Frozen design date: 2026-08-24

Archival purpose: This is the long-form reconstruction document. It intentionally preserves more detail than the compact baseline and includes the exact extracted text of the frozen compact architecture as an appendix.

1. Architectural status and claim boundary

Status: NORMATIVE unless explicitly marked otherwise.

Z0-A is a frozen design baseline, not a production-readiness certificate. Implementation claims are promoted only by claim-specific evidence.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: design freeze is separate from deployment; no autonomous-evolution claim; no biological or consciousness claim; evidence status remains independently versioned. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

2. Final architectural ruling

Status: NORMATIVE unless explicitly marked otherwise.

The source of truth must remain outside replaceable cognition, provider sessions, interfaces, telemetry implementations, and development agents.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: accepted change moves known canonical state to new canonical state; explicit authority; commit-time CAS; intent-bound idempotency; append-oriented accepted history. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

3. Five-plane topology

Status: NORMATIVE unless explicitly marked otherwise.

The normative topology contains exactly Canonical Spine, Operational, Observation, Development/Mason, and Evidence/Governance.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: Cognition is an Operational service; Ingress is an Operational service; Observation is not canonical truth; Mason is not a promotion authority. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

4. Canonical Spine ownership

Status: NORMATIVE unless explicitly marked otherwise.

The Spine owns accountable identity, accepted event history, canonical heads and revisions, checkpoint references, authorizations, and activation history.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: models cannot write directly; telemetry cannot rewrite state; commits require governed interface; history corrections are new events. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

5. Operational Plane

Status: NORMATIVE unless explicitly marked otherwise.

Operational services perform work: tasks, roles, role bindings, cognition routing, context loading, connectors, outbox dispatch, and effect settlement.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: provider identifiers are provenance; workflow engines are not source of truth; task state must survive process loss; effects remain explicit obligations. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

6. Observation Plane

Status: NORMATIVE unless explicitly marked otherwise.

Observation receives raw telemetry and independently produces attributed aggregate measurements, SLO results, invariant results, and gap signals.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: loss of telemetry must not break replay; detector version recorded; windowing recorded; Mason cannot choose its own evidence window. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

7. Development / Mason Plane

Status: NORMATIVE unless explicitly marked otherwise.

Mason is a proposal-only development capability that consumes valid gaps and produces reversible candidate phenotype revisions.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: cannot alter objectives; cannot alter hard constraints; cannot alter authority or identity; cannot self-evaluate; cannot self-promote. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

8. Evidence / Governance Plane

Status: NORMATIVE unless explicitly marked otherwise.

Evidence and Governance own claim predicates, falsifiers, evidence references, evaluator receipts, approval policy, promotion gates, and audit.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: PASS does not propagate to unrelated claims; OPEN means required evidence is absent; human approval is not the only security boundary; promotion is separately authorized. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

9. World object

Status: NORMATIVE unless explicitly marked otherwise.

World is the top-level governance and namespace boundary, not a model session or a single prompt-defined agent.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry

component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: contains constitutional invariants; contains canonical identifier namespace; can contain multiple entities and societies; tracks architecture/runtime lineage. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

10. Entity model

Status: NORMATIVE unless explicitly marked otherwise.

An Entity is a persistent accountable object with stable canonical identity, lifecycle, state head, authority ceiling, and event lineage.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: entity survives provider replacement; entity is not a role name; entity can be individual or organizational; retirement does not silently recycle identity. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

11. Society / company-001

Status: NORMATIVE unless explicitly marked otherwise.

company-001 is modeled as an operational Entity/Society with independent history, objectives, roles, tasks, policies, and resources.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: society can host multiple roles; society identity persists across executors; role changes do not fork society identity; organizational state is canonical where required. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

12. Role model

Status: NORMATIVE unless explicitly marked otherwise.

A Role is a responsibility slot with a versioned contract; it is not automatically an independent identity-bearing entity.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: examples: secretary, sales, accountant; role name does not mint authority; role can require skills; role output contract is versioned. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

13. RoleBinding

Status: NORMATIVE unless explicitly marked otherwise.

RoleBinding is the governed relationship that activates a Role for a holder or executor within scope and bounded grants.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: provider-neutral binding; scope and validity explicit; authority grant bounded by ceilings; binding changes are auditable. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

14. Holder and executor

Status: NORMATIVE unless explicitly marked otherwise.

The current holder/executor may be a human proxy, LLM-backed runtime, deterministic service, shared runtime, or dedicated entity.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: holder is operational provenance; holder replacement need not replace entity; holder capabilities remain bounded; credentials are scoped. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

15. Task object and Task Bus

Status: NORMATIVE unless explicitly marked otherwise.

A Task is a stable governed work unit with identity, state, role/scope, artifacts, approvals, outputs, and provenance; a durable bus coordinates handoff.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: task may cross providers; workflow engine state is not authoritative; retries preserve task identity; terminal/waiting states are explicit. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

16. Skill model

Status: NORMATIVE unless explicitly marked otherwise.

A Skill is a reusable capability contract and remains a library/runtime capability unless independent identity, authority, state, or lifecycle is justified.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: skills do not mint authority; repeated behavior may become deterministic skill; role may bind skills; entity birth is a separate governance decision. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

17. Provider-independent identity

Status: NORMATIVE unless explicitly marked otherwise.

Provider, model, session, thread, and channel identifiers are provenance only; canonical identity must not be derived from them.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: provider handoff preserves World/Entity/Role/Task IDs; provider switch may be recorded as provenance; identity change requires canonical event; session closure is not identity death. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

18. Objective Contract

Status: NORMATIVE unless explicitly marked otherwise.

Goals and hard boundaries are externalized into immutable, versioned canonical Objective Contracts rather than buried in prompts.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: objective; success metrics and SLOs; hard constraints; authority ceiling; risk class; observation contract; allowed adaptation surface; promotion policy. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

Objective Contract reference schema

```
{
  "objective_contract_id": "obj-...",
  "version": 1,
  "scope": "world|entity|role|task",
  "objective": "...",
  "success_metrics_and_slos": [],
  "hard_constraints": [],
  "authority_ceiling": [],
  "risk_class": "...",
  "observation_contract": {},
  "allowed_adaptation_surface": [],
  "promotion_policy": {},
  "created_by": "...",
  "created_at": "...",
  "previous_version": null
}
```


Phenotype Revision reference schema

```
{
  "phenotype_revision_id": "rev-...",
  "parent_revision_id": "rev-...",
  "objective_contract_id": "obj-...",
  "objective_version": 1,
  "change_set": [],
  "migration_plan": {},
  "rollback_plan": {},
  "expected_metrics": [],
  "risk": {},
  "evidence_requirements": [],
  "proposal_origin": "human|mason|other",
  "status": "CANDIDATE"
}
```

Canonical Event reference schema

```
{
  "event_id": "evt-...",
  "world_id": "world-001",
  "entity_id": "company-001",
  "event_type": "...",
  "event_version": 1,
  "occurred_at": "...",
  "committed_at": "...",
  "actor_principal": "...",
  "authority_ref": "...",
  "task_id": "...",
  "objective_ref": {"id": "obj-...", "version": 1},
  "expected_head": "...",
  "previous_head": "...",
  "idempotency_key": "...",
  "intent_hash": "...",
  "payload": {},
  "payload_hash": "...",
  "event_hash": "...",
  "fencing_token": 42,
  "evidence_refs": []
}
```

Gap Signal reference schema

```
{
  "gap_id": "gap-...",
  "objective_ref": {"id": "obj-...", "version": 1},
  "metric_or_invariant_id": "...",
  "window": {"start": "...", "end": "..."},
  "detector_version": "...",
  "observed_value": "...",
  "expected_condition": "...",
  "severity": "...",
  "telemetry_refs": [],
  "provenance": {},
  "status": "OPEN"
}
```

Evaluation Receipt reference schema

```
{
  "evaluation_receipt_id": "eval-...",
  "evaluator_principal_id": "...",
  "evaluator_role_or_credential_ref": "...",
  "evaluator_build_digest": "...",
  "candidate_revision_id": "...",
  "objective_ref": {"id":"obj-...","version":1},
  "test_suite_digest": "...",
  "environment_digest": "...",
  "result": "PASS|FAIL|INCONCLUSIVE",
  "metrics": {},
  "evidence_refs": [],
  "evaluated_at": "...",
  "attestation_or_signature": null
}
```

19. Constitutional change path

Status: NORMATIVE unless explicitly marked otherwise.

Changing Objective, Hard Constraints, Authority ceilings, identity rules, or constitutional invariants is not an ordinary Mason optimization.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: separate governance authorization; stronger evidence/approval; versioned change; rollback/transition plan; audit trail. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

20. Phenotype Revision

Status: NORMATIVE unless explicitly marked otherwise.

A Phenotype Revision is a versioned candidate execution/configuration change linked to a parent revision and a specific Objective version.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: change set; migration plan; rollback plan; expected metrics; risk; evidence requirements; candidate status before activation. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

21. Authority algebra

Status: NORMATIVE unless explicitly marked otherwise.

Effective authority is the intersection of the Entity ceiling, active RoleBinding grant, optional Task grant, policy allowances, and temporal/scope validity.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: edges are conditions not grants; role names do not grant; authority expansion is privileged; fail closed on ambiguity. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

22. Authorization record

Status: NORMATIVE unless explicitly marked otherwise.

Every privileged canonical change or external effect should point to an authorization record or derivable authority context.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: principal identity; scope; required capability; grant source; expiry; risk class; policy decision. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

23. Canonical event schema

Status: NORMATIVE unless explicitly marked otherwise.

Canonical events bind semantic content, lineage, authority, objective/revision context, concurrency preconditions, and provenance.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: event_id unique; event_version explicit; expected/previous head; intent hash; payload hash; fencing token; evidence refs. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

24. Commit-time CAS

Status: NORMATIVE unless explicitly marked otherwise.

The expected head must be checked inside the same transaction that advances the canonical head; application pre-read is insufficient.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: two writers one head: only one commits; stale writer fails closed; no silent overwrite; retry requires new proposal or rebase. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

25. Intent-bound idempotency

Status: NORMATIVE unless explicitly marked otherwise.

Idempotency keys are bound to semantic intent; identical retries may deduplicate, while key reuse with changed semantics is a collision error.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: durable idempotency record; semantic hash; target/type included; collision is explicit. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

26. Atomic canonical transition

Status: NORMATIVE unless explicitly marked otherwise.

Event append, head advance, required authoritative projections, and outbox obligation creation are atomically coupled for a governed transition.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: fault injection must not create half-state; transaction rollback on partial failure; analytical projections may be asynchronous; authoritative minimum remains atomic. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

27. Transactional Outbox

Status: NORMATIVE unless explicitly marked otherwise.

External side effects are represented as obligations committed with the canonical intent and later claimed by an effect executor.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: obligation has stable id; attempts audited; provider id captured; ambiguous outcome reconciled. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

28. Effect Receipt

Status: NORMATIVE unless explicitly marked otherwise.

Effect success or externally visible settlement is recorded by a receipt that links the obligation, attempt, connector, provider result, and reconciliation state.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: scoped effectively-once only; no universal exactly-once claim; duplicate protection connector-specific; irreversible ambiguity is first-class. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

29. Sequencer lease

Status: NORMATIVE unless explicitly marked otherwise.

A sequencer lease identifies the current serialization authority for a governed scope and has explicit expiry and renewal rules.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: lease state durable for claim scope; single active authority or equivalent serialization; expired holder cannot be trusted by itself; monitor lease health. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

30. Fencing token

Status: NORMATIVE unless explicitly marked otherwise.

Each sequencer epoch receives a monotonically increasing fencing token so stale processes are rejected even if they remain alive.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: storage-side validation; stale token rejection; fresh token after failover; fresh higher token after restore. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

31. DB-enforced append-only

Status: NORMATIVE unless explicitly marked otherwise.

Committed history is protected by database privileges and controls, not only by application convention.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: runtime roles denied UPDATE; denied DELETE; denied TRUNCATE; correction is a new event; break-glass admin is audited. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

32. Canonicalization and hashes

Status: NORMATIVE unless explicitly marked otherwise.

Semantic hashes use deterministic serialization and versioned hash formulas to make content and chain verification reproducible.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: RFC 8785/JCS is a suitable reference; hash binds previous head and semantic content; hash is tamper-evidence; hash is not confidentiality or authorization. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

33. Checkpoint model

Status: NORMATIVE unless explicitly marked otherwise.

A checkpoint is a materialized state bound to a known canonical head and reducer/schema version; it accelerates recovery but does not replace accepted history.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: checkpoint hash/reference; version compatibility; periodic verification; stale checkpoints detected. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

34. Verified replay

Status: NORMATIVE unless explicitly marked otherwise.

Replay verifies chain continuity, hashes, event versions, duplicate identifiers, ordering, reducer compatibility, and final head before materialization.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: corruption fails closed; mutation removing hash check must be caught; duplicate event id rejected; unsupported version requires migration path. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

35. Clean-host restore

Status: NORMATIVE unless explicitly marked otherwise.

Continuity is not proven by restarting the same process. Recovery must reconstruct on a clean environment and then accept a fresh governed write.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: different system identity; restore schema/data; verify chain/head/objective/revision; recreate separated credentials; fresh lease/fencing; record RTO/RPO and restore receipt. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

36. Ingress and routing

Status: NORMATIVE unless explicitly marked otherwise.

External input is normalized, authenticated where needed, resolved to entity/task/role scope, and routed through Operational services without direct truth mutation.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: channel-specific data stays provenance; routing decision auditable; unknown identity handled explicitly; input cannot bypass policy. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

37. Brain Gateway

Status: NORMATIVE unless explicitly marked otherwise.

A provider-neutral Brain Gateway invokes replaceable cognition using scoped context, role contracts, tool descriptions, authority limits, and output schemas.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: ChatGPT/Grok/DeepSeek/Gemini/Claude/local/deterministic possible; provider selection by policy; brain output is proposal/result; brain cannot directly commit. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

38. Context Compiler K0-K4

Status: NORMATIVE unless explicitly marked otherwise.

Context is compiled by scope: World kernel, Entity/Society context, Role contract, current Task, and minimum provenance-bearing retrieved evidence.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: minimize context without losing invariants; source remains outside generated summary; staleness tracked; provider context window is not canonical memory. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

39. Memory architecture

Status: NORMATIVE unless explicitly marked otherwise.

Retrieval memory is separated from authoritative state and may include working, episodic, semantic, document/evidence, cold archive, and protected vault layers.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: summary does not replace source; vector index is rebuildable projection; provenance and timestamps retained; retention/privacy policy explicit. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

40. Observation pipeline

Status: NORMATIVE unless explicitly marked otherwise.

The normative pipeline is raw telemetry -> normalization -> aggregation/windowing -> SLO/invariant evaluation -> gap signal -> evidence registry.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: detector outside Mason; source refs retained; window and method recorded; coverage gaps explicit. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

41. SLO and invariant contracts

Status: NORMATIVE unless explicitly marked otherwise.

Success metrics and invariants are versioned as part of Objective/Observation contracts so optimization is judged against a stable target.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: threshold changes are versioned; identity continuity can be invariant; authorization-before-effect can be invariant; restore success can be SLO/gate. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

42. Gap Signal

Status: NORMATIVE unless explicitly marked otherwise.

A gap is an attributed discrepancy with objective version, metric/invariant, window, detector version, observed value, expected condition, and evidence refs.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: status OPEN/CLOSED/INVALIDATED/SUPERSEDED/EXPIRED; Mason does not own gap definition; confidence/severity optional but explicit; gap provenance immutable. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

43. Mason proposal contract

Status: NORMATIVE unless explicitly marked otherwise.

Mason references a valid gap and current objective, proposes the smallest reversible intervention, supplies tests and rollback, and stays in the adaptation surface.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: proposal has parent revision; proposal has risk; proposal has expected metrics; forbidden fields rejected before evaluation. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

44. Evaluator identity

Status: NORMATIVE unless explicitly marked otherwise.

Evaluator identity must be attributable to an authenticated principal and exact evaluator build/profile; database constraints alone are insufficient proof.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: principal id; credential/role ref; build digest; candidate/objective refs; environment/test digests; metrics/evidence; optional attestation/signature. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

45. Evaluation Receipt

Status: NORMATIVE unless explicitly marked otherwise.

An Evaluation Receipt is the durable record that binds the evaluator, candidate, objective, test environment, result, metrics, and evidence.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: PASS/FAIL/INCONCLUSIVE explicit; receipt immutable once committed; promotion references receipt; receipt does not store secrets. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

46. Separation of duties

Status: NORMATIVE unless explicitly marked otherwise.

Observation measures, Mason proposes, Evaluator judges, Promotion Authority promotes, Canonical Spine records, and Effect Executor settles obligations.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: separate credentials for high assurance; single host may contain logical modules only at low evidence levels; self-approval forbidden; one actor controlling all stages violates Z0-A. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

47. Promotion gate

Status: NORMATIVE unless explicitly marked otherwise.

A candidate can become active only after evidence and policy checks and a separately authorized promotion decision committed with CAS.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry

component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: receipt required; hard constraints unchanged; authority unchanged unless constitutional path; stale active revision rejected; activation itself is canonical event. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

48. Rollback

Status: NORMATIVE unless explicitly marked otherwise.

Rollback creates a new canonical event activating a known-good or repair revision; it never erases the failed history.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: trigger recorded; incident/evidence refs; authority recorded; post-rollback verification; external effect compensation handled separately. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

49. Evidence levels E0-E5

Status: NORMATIVE unless explicitly marked otherwise.

Evidence levels separate definition, static artifacts, local executable evidence, persistent integration, production-like testing, and longitudinal operation.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: E0 defined; E1 static; E2 local/disposable; E3 persistent/integration; E4 restore/security/load/chaos/independent credentials; E5 longitudinal SLO/failure envelope. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

50. Proof Registry

Status: NORMATIVE unless explicitly marked otherwise.

Every major claim has its own predicate, falsifier, required evidence level, status, environment/build information, and evidence references.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: OPEN is not failure but absence of required proof; runner output should drive status; manual status cannot override evidence; claims do not inherit neighbor PASS. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

51. Threat model

Status: NORMATIVE unless explicitly marked otherwise.

The architecture assumes failures and adversaries across providers, credentials, concurrency, operators, telemetry, storage, and external effects.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: stale writer; compromised model; prompt injection; compromised Mason; evaluator substitution; credential theft; partial DB failure; corrupt backup; operator error. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

52. Secrets and credentials

Status: NORMATIVE unless explicitly marked otherwise.

Secret material is kept outside canonical event payloads and evaluation receipts; only references and principals belong in durable audit records.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: rotation does not change identity; logs scanned for leaks; least privilege; high-risk credentials separated. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

53. Database role matrix

Status: NORMATIVE unless explicitly marked otherwise.

Reference PostgreSQL roles separate sequencer, observer, Mason, evaluator, effect executor, promotion authority, audit, and migration capabilities.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: GRANT/REVOKE versioned; negative privilege tests; runtime cannot rewrite events; evaluator cannot promote; effect executor cannot self-authorize. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

54. Service/API boundaries

Status: NORMATIVE unless explicitly marked otherwise.

Logical service boundaries include commit, query/projection, task, brain gateway, context compiler, observation, gap detection, Mason, evaluation, promotion, effects, and recovery.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: not required to be microservices; contracts matter more than process count; capability boundaries explicit; API versions governed. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

55. Lifecycle states

Status: NORMATIVE unless explicitly marked otherwise.

Entity lifecycle is an explicit operational state machine rather than an implicit property of whether a model session is open.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: PROVISIONED; ACTIVE; DEGRADED; FROZEN/QUARANTINED; HIBERNATED; RESTORED; RETIRED/TOMBSTONED. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

56. Health, repair, sleep and wake

Status: NORMATIVE unless explicitly marked otherwise.

Earlier World concepts of health, sleep/wake, repair, freeze, and reconstruction are retained only when mapped to explicit transitions and evidence.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: sleep can reduce active cognition; wake reloads current canonical state; repair is governed revision/recovery; no biological claim. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

57. Multi-brain routing

Status: NORMATIVE unless explicitly marked otherwise.

Multiple cognition providers may sit behind Brain Gateway and be selected by capability, cost, latency, privacy, jurisdiction, availability, or policy.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: fallback preserves identity; ensemble vote is not authority; provider metadata retained as provenance; hard constraints cannot weaken on fallback. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

58. Skill to Role to Entity growth

Status: NORMATIVE unless explicitly marked otherwise.

Repeated successful behavior may become a deterministic skill, persistent specialization may justify a role, and independent isolation/state may justify an entity.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: growth is evidence-driven; Mason cannot autonomously birth entities in Z0-A; authority ceilings inherited/bounded; genome/evolution language remains exploratory. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

59. Effect risk classes

Status: NORMATIVE unless explicitly marked otherwise.

External actions are classified by reversibility and impact so authorization and evidence requirements scale with risk.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: read-only; reversible low-risk write; externally visible communication; financial/legal/contractual; destructive/high-impact. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

60. Audit model

Status: NORMATIVE unless explicitly marked otherwise.

Audit must reconstruct who or what requested which action, under which objective and authority, against which head, with which provider/build, and with what result.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: link events/tasks/principals; link objective/revision; link effects/receipts; link gaps/evaluations/promotions; survives workflow engine loss. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

61. Failure-response matrix

Status: NORMATIVE unless explicitly marked otherwise.

Failure handling is explicit and fail-closed where ambiguity could corrupt identity, authority, history, or irreversible effects.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: stale writer -> reject; collision -> reject; partial transaction -> rollback; telemetry outage -> coverage degraded; hash mismatch -> stop replay; ambiguous effect -> reconcile. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

62. Adversarial testing

Status: NORMATIVE unless explicitly marked otherwise.

Tests intentionally attack boundaries through races, stale tokens, duplicate identifiers, privilege violations, corrupted replay, forbidden Mason changes, and stale promotion.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: negative tests mandatory; fault injection; cross-provider continuity test; restore on clean host; effect ambiguity test. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

63. Mutation testing

Status: NORMATIVE unless explicitly marked otherwise.

Mutation gates verify that removing a critical check such as replay hash verification or idempotency collision handling causes tests to fail.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: avoid vacuous test counts; report mutation score by family; critical mutants cannot survive; runner/evidence registry linked. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

64. Deployment profiles

Status: NORMATIVE unless explicitly marked otherwise.

Evidence levels map to deployment profiles ranging from local development to persistent integration, production-like separated credentials, and longitudinal operation.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: E2 local; E3 persistent integration; E4 restore/security/load/chaos; E5 longitudinal. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

65. Backup architecture

Status: NORMATIVE unless explicitly marked otherwise.

Backups cover canonical database, event history, schemas/migrations, objective/revision artifacts, evidence registry, manifests, and protected recovery procedures.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: backup is not proof until restore tested; checksums and manifests; restore order documented; source code alone is insufficient. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

66. Repository and release discipline

Status: NORMATIVE unless explicitly marked otherwise.

Normative artifacts are versioned and stable releases are immutable; fixes create new versions, manifests, and hashes.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: no secrets/private payloads in release; schema migrations versioned; rollback plan for destructive change; machine-readable contracts accompany prose. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

67. Lineage from World v6.2

Status: NORMATIVE unless explicitly marked otherwise.

World v6.2 contributed entity stability, replaceable brains, scoped context, authority outside cognition, effect handling, evidence gates, and recovery discipline.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: retained where compatible; older topology not automatically current; fractal/multi-brain ideas remain operational patterns; Z0-A is stricter about five planes. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

68. Lineage from World 7

Status: NORMATIVE unless explicitly marked otherwise.

World 7 contributed persistent identity, Spine semantics, proposal-only cognition, expected-head conflicts, idempotency, hash chains, reconstruction, and explicit proof obligations.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: genomic metaphor moved out of normative core; autonomous evolution claim rejected without evidence; review exposed vacuous/self-referential tests; mutation and registry discipline strengthened. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

69. Lineage from World 8 v0.1/v0.1.1

Status: NORMATIVE unless explicitly marked otherwise.

World 8 v0.1/v0.1.1 provides operational lineage for world-001, company-001, RoleBindings, Task Bus, and persistent canonical history.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: prior evidence remains valuable; new Z0-A gates require new evidence; Observation/Mason split is new fixed boundary; evaluator attribution is strengthened. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

70. Superseded interpretations

Status: NORMATIVE unless explicitly marked otherwise.

Earlier interpretations that conflict with Z0-A are preserved only for history and must not silently return as current architecture.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: more than five normative planes; Role treated as Entity by name; Mason-controlled telemetry/gaps; direct self-promotion; objective only in prompt; hash chain called secure storage. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

71. Explicit non-claims

Status: NORMATIVE unless explicitly marked otherwise.

The architecture does not claim consciousness, biological life, AGI, legal personhood, proven autonomous evolution, universal exactly-once effects, or production readiness by design freeze alone.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: claim modesty is normative; metaphors are operational only; production requires E3/E4/E5 as applicable; security claims remain scoped. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

72. Formal safety properties

Status: NORMATIVE unless explicitly marked otherwise.

Core claims are expressed as falsifiable safety properties so architecture can be tested rather than defended by metaphor.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry

component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: provider replacement preserves identity; stale head cannot commit; stale fencing cannot commit; Mason cannot mutate hard boundaries; promotion requires attributed receipt; restore requires fresh write. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

73. Liveness goals

Status: NORMATIVE unless explicitly marked otherwise.

The system should eventually resolve valid tasks, replace expired leases, settle or reconcile effects, decide candidates, recover entities, and close or invalidate gaps.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: liveness never bypasses safety; timeouts become explicit states; stuck work is observable; manual escalation is allowed but audited. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize

storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

74. Reference end-to-end scenario

Status: NORMATIVE unless explicitly marked otherwise.

A representative proof crosses all five planes: persistent task, provider handoff, governed commit, real low-risk effect receipt, SLO breach, Mason proposal, independent evaluation, and separate promotion.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: same entity/task identity across provider change; effect authorization/outbox/receipt; gap from predefined SLO; revision activated with CAS. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed

evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

75. Z0-A execution roadmap

Status: NORMATIVE unless explicitly marked otherwise.

Implementation proceeds from contract freeze through Spine hardening, Entity/Role separation, Objective/Revision versioning, Observation, Mason, evaluation, promotion, recovery, and operational proof.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: Z0A-0 contracts; Z0A-1 spine; Z0A-2 identity/role; Z0A-3 objective/revision; Z0A-4 observation; Z0A-5 mason; Z0A-6 evaluator; Z0A-7 promotion; Z0A-8 restore; Z0A-9 operational proof. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

76. Z0-A exit gates

Status: NORMATIVE unless explicitly marked otherwise.

Architecture and executable boundary must agree before a gate is called closed; document-only compliance is insufficient.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: G1 plane isolation; G2 entity/role split; G3 objective/revision versioning; G4 observation independence; G5 Mason confinement; G6 evaluator identity; G7 spine atomicity; G8 recovery; G9 effects. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

77. Future-maintainer reconstruction runbook

Status: NORMATIVE unless explicitly marked otherwise.

A future maintainer starts from the frozen architecture version, verifies manifests and schemas, locates proof and role matrices, replays state, checks adversarial gates, and only then resumes development.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: do not trust old diagrams over current manifest; verify active objective/revision; verify evaluator attribution; verify latest restore; verify provider handoff and effect reconciliation. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

78. Minimum machine-readable artifact set

Status: NORMATIVE unless explicitly marked otherwise.

The prose must be accompanied by schemas, role matrices, forbidden-transition policy, promotion policy, proof registry, runtime modules, observation modules, evidence, and release manifests.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: Objective schema; Revision schema; Event schema; Gap schema; Evaluation receipt schema; RoleBinding schema; Outbox/effect schema; DB role matrix; proof registry. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

79. Final continuity contract

Status: NORMATIVE unless explicitly marked otherwise.

Cognition and interfaces are replaceable; accountable identity, canonical state, authority, objectives, accepted history, and evidence remain governed and persistent.

Scope and ownership

This part of the architecture exists to make ownership and trust boundaries explicit. The relevant canonical or governed objects must carry stable identifiers and version references where change over time matters. Runtime convenience state may be cached or projected, but it cannot silently redefine the authoritative object. Where a provider, workflow engine, model call, UI, or telemetry component participates, its identity is retained as provenance rather than promoted into canonical identity.

Normative rules

The required properties for this area are: prose and runtime must agree; no bypass path; all stronger claims remain evidence-gated; future revisions require explicit governance. These requirements are interpreted fail-closed when ambiguity can alter identity, authority, accepted history, objective boundaries, revision activation, or an irreversible external effect. A later implementation may optimize storage, batching, caching, or service decomposition, but it may not change the semantic boundary without a versioned architecture decision.

Forbidden shortcuts and failure interpretation

A successful happy-path demonstration is not sufficient if a bypass path can violate the rule. Direct writes from replaceable cognition, silent overwrite of stale state, unversioned objective changes, self-approval, missing provenance, or evidence inherited from an unrelated claim are treated as architectural failures. If the implementation cannot establish the required property at its claimed evidence level, the corresponding Proof Registry entry remains OPEN or FAIL; the prose does not upgrade it.

Evidence obligation

Evidence should include the narrowest reproducible test that can falsify the claim, plus environment/build identity and artifact references. At E3 and above, persistent storage, independent processes or credentials where relevant, failure injection, and recovery behavior are tested on the target integration rather than inferred from local mocks.

Appendix A - Reference commit algorithm

```
BEGIN TRANSACTION
  authenticate principal
  load/verify authorization context
  verify active sequencer lease
  verify fencing_token >= current_fence and belongs to active epoch
  verify expected_head == canonical_head
  validate schema/event version
  compute semantic intent hash
  check idempotency record
    if same key + same intent: return prior committed result
    if same key + different intent: IDEMPOTENCY_COLLISION
  append canonical event
  advance canonical head
  update required authoritative projections
  create outbox obligation if external effect is planned
COMMIT
```

No application pre-read can replace the in-transaction expected-head predicate. No provider/model response can bypass authority and commit rules.

Appendix B - Separation-of-duties matrix

Capability	Allowed	Forbidden
Promotion Authority / Human Root	Approve or reject gated promotion	Rewrite history or replace technical security controls
Sequencer	Commit with valid lease, fencing and CAS	Self-approve development changes
Observer	Measure and emit attributed signals	Mutate canonical state or redefine objective for its own measurements
Mason	Propose candidate revisions	Self-evaluate, self-promote, change hard boundaries
Evaluator	Judge candidate and emit receipt	Promote the same candidate
Effect Executor	Settle authorized obligations	Create its own authorization
Brain Provider	Produce cognition/proposals/results	Own canonical identity or direct

		canonical commit
--	--	------------------

Appendix C - Mandatory negative tests

1. Two writers with the same expected head: exactly one commits.
2. Same idempotency key and same intent: prior result is replayed/deduplicated.
3. Same key and changed intent: explicit collision error.
4. Stale fencing token: rejected.
5. Fault between event append/head/outbox: transaction leaves no half-state.
6. Runtime role UPDATE/DELETE/TRUNCATE on committed events: denied.
7. Replay with modified event payload/hash: rejected.
8. Duplicate event_id during replay: rejected.
9. Mutation that disables hash verification: test suite must fail.
10. Mutation that removes idempotency collision handling: test suite must fail.
11. Mason attempts Objective/Hard Constraint/Authority/Identity/Spine change: rejected.
12. Mason attempts promotion: rejected.
13. Promotion without attributed evaluation receipt: rejected.
14. Promotion against stale current revision/head: rejected.
15. Evaluator receipt with missing or mismatched build/principal: does not satisfy gate.
16. Effect executor attempts effect without committed obligation: rejected.
17. Ambiguous provider timeout: reconciliation path, not blind duplicate.
18. Clean-host restore: chain/revision verified and one fresh governed write succeeds.
19. Provider handoff: same World/Entity/Role/Task canonical identity remains.
20. Telemetry outage: observation marked degraded; no fabricated gap is produced.

Appendix D - Minimum repository structure

```

/contracts/
  OBJECTIVE_CONTRACT.schema.json
  PHENOTYPE_REVISION.schema.json
  CANONICAL_EVENT.schema.json
  ROLE_BINDING.schema.json
  GAP_SIGNAL.schema.json
  EVALUATION_RECEIPT.schema.json
  OUTBOX_OBLIGATION.schema.json
  EFFECT_RECEIPT.schema.json
/governance/
  DB_ROLE_MATRIX.yaml
  FORBIDDEN_TRANSITIONS.yaml
  PROMOTION_POLICY.yaml
  PROOF_REGISTRY.json
/runtime/
  sequencer/
  replay/
  task_bus/
  brain_gateway/
  context_compiler/
  effect_executor/
/observation/
  telemetry_schema/
  aggregators/
  detectors/
/development/

```

```
mason/  
/evidence/  
  unit/  
  integration/  
  mutation/  
  restore/  
  provider_handoff/  
  effects/  
/releases/  
  manifest.json  
  SHA256SUMS
```

Appendix E - Two-year reconstruction procedure

A maintainer returning after a long interruption should not begin by running the newest model or opening an old workflow. First resolve the architecture version from the release manifest. Then verify schemas, database role matrix, current canonical head, active Objective Contract, active Phenotype Revision, Proof Registry, and the latest restore receipt. Perform replay verification, concurrency/idempotency/fencing tests, Mason negative tests, evaluator-attribution checks, provider-handoff checks, and effect reconciliation checks. Only after those foundations agree should new development resume.

Appendix F - Status of retained World 7 vocabulary

The earlier terms genome, phenotype, development, genesis, evolution, sleep, repair, and non-extinction are not erased from project history. In Z0-A, only operational meanings that map to explicit contracts are retained. Phenotype means a versioned expressed runtime revision. Repair means a governed revision or recovery transition. Sleep/wake are lifecycle operations. Genome/evolution are exploratory until population, heritable variation, external fitness, selection, and independent empirical evidence exist. No biological-life claim is implied.

Appendix G - Frozen baseline source files

The exact compact baseline PDF is included unchanged in the archival package and appended to the final archival PDF. The TXT/MD master also retains the extracted text snapshot.

World 8 - Z0-A Final Architecture Baseline

Provider-independent persistent runtime with independent observation and proposal-only development

Status: FINAL DESIGN BASELINE / NOT PRODUCTION

Frozen design date: 2026-08-24

Scope: Normative Z0-A architecture. Implementation and evidence promotion remain separately gated.

Central rule	Cognition and interfaces are replaceable; accountable identity, canonical state, authority, objectives, and accepted history are not.
Fixed topology	Five planes: Canonical Spine, Operational, Observation, Development/Mason, Evidence/Governance.
Mason boundary	Proposal only. No direct change to Objective, Hard Constraints, Authority, Identity, Canonical Spine, or Promotion.
Claim boundary	This document freezes the Z0-A design. It does not claim production readiness, autonomous evolution, or full autonomy.

1. Final architectural ruling

World 8 Z0-A is a persistent, auditable runtime whose source of truth remains outside replaceable model providers, sessions, user interfaces, telemetry systems, and development agents. Every accepted change must move from a known canonical state to a new canonical state through explicit authority, commit-time CAS, intent-bound idempotency, append-oriented history, and claim-specific evidence.

2. The five fixed planes

Plane	Owns	Must not become
Canonical Spine	World/Entity identity, event history, head/version, checkpoints, authorizations, canonical revisions	A model/UI/telemetry-controlled store
Operational	Entity/Role/RoleBinding, Task, ingress, Brain Gateway, context, outbox/effects/connectors	A provider-owned identity domain
Observation	Raw telemetry, normalization, aggregation, SLO evaluation, gap signals	Canonical truth or a Mason-controlled detector
Development / Mason	Candidate Phenotype Revision proposals	Self-authorizing/self-promoting evolution
Evidence / Governance	Proof Registry, evaluator receipts, approvals, policy, promotion gates, audit	A mechanism that inherits evidence between unrelated claims

3. Entity, Role, and RoleBinding

Entity and Role are distinct. **company-001** is an operational Entity/Society. **secretary-role** is a responsibility slot activated through a governed **role_binding**. Provider/model/session identifiers are provenance, not canonical object identity.

4. Versioned Objective Contract and Phenotype Revision

The Objective Contract is a canonical, versioned contract containing objective, success metrics/SLOs, hard constraints, authority ceiling, risk class, observation contract, allowed adaptation surface, and promotion policy. A Phenotype Revision is a versioned candidate execution change referencing its parent revision and objective version, with explicit migration, rollback, metrics, risk, and evidence.

Objective Contract -> constrains Observation and allowed adaptation
Phenotype Revision -> proposes a reversible runtime change
Constitutional change -> separate governance path for Objective/Hard Constraints/Authority/Identity

5. Observation precedes Mason

```
raw telemetry -> normalization -> aggregation/windowing -> SLO/invariant evaluation ->
gap_signal -> Evidence Registry -> Mason may consume
```

Mason may not select the evidence set, redefine the SLO, or manufacture a gap. Gap detection may be deterministic or may use an independently versioned classifier, but that detector remains outside Mason and must be independently testable.

6. Mason is proposal-only

- Mason references a valid gap and current Objective Contract version.
- It proposes a minimal reversible change with tests and rollback.
- It cannot modify Objective, Hard Constraints, Authority, Identity, or Canonical Spine.
- It cannot evaluate or promote its own candidate.
- It cannot directly write canonical state.

7. Independent evaluator identity

Database constraints prove allowed database capabilities, not the real identity/version of the evaluator that produced a result. Each Evaluation Receipt therefore binds evaluator principal, role/credential reference, evaluator build digest, candidate revision, objective version, test-suite/environment digest, result metrics, evidence references, and attestation/signature where available. Mason, evaluator, sequencer, effect executor, and promotion authority use separate capabilities and credentials.

8. Canonical Spine commit contract

- **Commit-time CAS:** expected head is checked within the transaction that advances the head.
- **Intent-bound idempotency:** identical retry deduplicates; key reuse with changed semantics is a collision error.
- **Atomic append/head/outbox:** event append, head advance, permitted projections, and planned effects are transactionally coupled.
- **Transactional outbox + receipts:** effects are obligations; attempts and provider-side receipts settle them. Only scoped effectively-once semantics are targeted.
- **Sequencer lease + fencing:** stale/displaced sequencers cannot commit.
- **DB-enforced append-only history:** runtime roles cannot UPDATE/DELETE/TRUNCATE committed events.
- **Verified replay:** chain, hashes, versions, duplicate IDs, and final head are checked before materialization.

- **Clean-host restore:** reconstruction must work from durable artifacts and accept one fresh governed post-restore write under a fresh lease/fencing token.

9. Provider-independent continuity and scoped context

A provider switch must preserve the same World/Entity/Society/Role/Task IDs and canonical lineage. Context loading is scoped K0-K4: World kernel, Society/Entity context, Role contract, current Task, and minimum provenance-bearing retrieved evidence.

10. Full runtime and development path

External input

- > Operational ingress/auth/route
- > scoped context + replaceable cognition
- > proposal/result
- > authority/policy evaluation
- > Canonical commit
- > task/state projection + optional outbox
- > effect attempt -> receipt
- > Observation telemetry
- > SLO evaluation -> gap_signal
- > Mason candidate revision
- > independent evaluation
- > Governance promotion decision
- > canonical revision activation

11. Evidence and governance discipline

Every claim carries its own predicate, falsifier, required evidence level, status, and evidence references. OPEN means required evidence does not exist. Neighboring PASS results do not upgrade an unrelated claim.

Level	Meaning
E0	Defined / untested
E1	Static contract/schema/privilege evidence
E2	Local/disposable executable evidence
E3	Persistent/integration evidence
E4	Production-like restore/security/load/chaos/independent credentials
E5	Longitudinal evidence under predefined SLO/failure envelope

12. Separation of duties

Capability	Allowed	Forbidden
Promotion Authority / Human Root	Approve/revoke gated promotion	Rewrite history or replace security controls
Sequencer	Commit with valid lease/fencing/CAS	Self-approve
Observer	Measure and emit attributed signals	Canonical mutation or objective definition
Mason	Propose candidate revision	Self-evaluate or promote
Evaluator	Judge candidate and emit receipt	Propose the same candidate or promote it
Effect Executor	Settle authorized obligations	Create its own authorization

13. Promotion and rollback

1. Observation records a valid gap.
2. Mason or a human proposer creates a candidate revision.
3. Independent shadow/replay/test evaluation runs.
4. An attributed Evaluation Receipt is committed.
5. Governance verifies hard constraints and authority are unchanged.
6. Promotion Authority decides.
7. Revision activation is committed with CAS against the current canonical revision/head.
8. Regression triggers a new rollback event to a known-good revision.

14. Explicit non-claims

- No claim of consciousness, biological life, AGI, or legal personhood.
- Mason is not proven autonomous evolution in Z0-A.
- Hash chains are tamper-evident, not privileged-storage security.
- Database constraints alone do not prove evaluator identity.
- Human approval alone is not a sufficient security boundary.
- No universal exactly-once external-effects claim.
- Production readiness remains gated on credential separation, persistent-target restore, security/load/chaos testing, and SLOs.

15. Z0-A exit gates

Gate	PASS criterion
------	----------------

Plane isolation	Five planes enforced in APIs/schema/DB roles with bypass tests.
Entity/Role split	Portable RoleBinding; provider metadata not required for canonical identity.
Objective/Revision versioning	Immutable/versioned contracts with CAS activation.
Observation independence	raw->aggregate->SLO->gap outside Mason with provenance.
Mason confinement	Negative tests prove it cannot change hard boundaries or promote.
Evaluator identity	Receipts bind principal/build/input/output and credentials are separated.
Spine atomicity	CAS/idempotency/append/head/outbox/fencing adversarial gates PASS.
Recovery	Clean-host restore plus fresh governed post-restore write PASS.
Effects	Authorization->outbox->attempt->receipt and duplicate/ambiguity tests PASS.

16. Lineage from World 8 v0.1.1

World 8 v0.1/v0.1.1 remains important operational evidence for world-001, company-001, RoleBindings, Task Bus, and persistent canonical history. Z0-A closes the next architectural layer by fixing the five-plane model, separating Observation from Mason, versioning Objective/Phenotype Revision, and making evaluator identity evidence explicit. Prior evidence does not automatically PASS these new Z0-A gates.